

- 页表创建失败时如何清理已分配的资源？
 - 如何检测和处理内存泄漏？
-

实验4：中断处理与时钟管理

实验目标

通过分析xv6的中断处理机制，理解操作系统如何响应硬件事件，实现完整的中断处理框架和时钟中断驱动的任务调度。

核心学习资料

RISC-V中断机制

- RISC-V特权级规范 第3章：Machine-Level ISA
 - 3.1.9节：Machine Interrupt Registers
 - 3.2.1节：Machine Timer Registers
- RISC-V特权级规范 第12章：Supervisor-Level ISA
 - 12.1.3节：Supervisor Interrupt Registers
 - 重点理解：mie、mip、sie、sip寄存器的作用

xv6中断处理源码分析

- `kernel/trap.c` - 中断和异常处理
 - 重点函数：`usertrap()`，`kerneltrap()`，`devintr()`
 - 学习要点：中断分发、异常处理、系统调用入口
- `kernel/kernelvec.S` - 内核态中断向量
 - 重点：上下文保存和恢复机制
- `kernel/start.c` - 机器模式初始化
 - 重点函数：timer中断的设置和代理

时钟管理理论

- SBI规范：<https://github.com/riscv-non-isa/riscv-sbi-doc>
 - 第4.6节：Timer Extension
- 操作系统概念 第5章：CPU调度

任务列表

任务1：理解RISC-V中断架构

学习重点：

1. 分析中断特权级委托：
 - Machine Mode → Supervisor Mode 委托
 - medeleg: 异常委托寄存器
 - mideleg: 中断委托寄存器
 - 为什么需要中断委托?
 - 哪些中断应该委托给S模式?
2. 理解中断寄存器组合：
 - mie/sie: 中断使能寄存器
 - mip/sip: 中断挂起寄存器
 - mtvec/stvec: 中断向量基址
 - mcause/scause: 中断原因寄存器

深入思考:

- 时钟中断为什么在M模式产生，却在S模式处理?
- 如何理解“中断是异步的，异常是同步的”?

任务2: 分析xv6的中断处理流程

代码阅读指导:

1. 研读 `start.c` 中的机器模式设置:

```
1 // 时钟中断委托给S模式
2 w_mideleg(r_mideleg() | (1L << 5));
3 // 设置机器模式陷阱向量
4 w_mtvec((uint64)timerverc);
```

- 为什么时钟中断需要特殊处理?
 - `timerverc` 的作用是什么?
2. 分析 `kernelvec.S` 的上下文切换：
 - 哪些寄存器需要保存?
 - 为什么不保存所有寄存器?
 - 栈的使用策略是什么?
 3. 理解 `trap.c` 的中断分发:

```
1 void kerneltrap(void) {
2     // 中断还是异常?
3     // 如何确定中断源?
4     // 如何调用相应处理函数?
5 }
```

关键问题:

- 中断处理中的重入问题如何解决?
- 中断处理时间过长会有什么后果?

任务3：设计你的中断处理框架

架构设计要求：

1. 设计中断向量表结构
2. 定义中断处理函数接口
3. 实现中断的注册和注销机制

设计考虑：

```
1 // 中断处理函数类型
2 typedef void (*interrupt_handler_t)(void);
3
4 // 中断控制接口
5 void trap_init(void); // 初始化中断系统
6 void register_interrupt(int irq, interrupt_handler_t h); // 注册中断处理函数
7 void enable_interrupt(int irq); // 开启特定中断
8 void disable_interrupt(int irq); // 关闭特定中断
9
10 // 你需要考虑的问题：
11 // 1. 如何设计中断优先级？
12 // 2. 是否支持中断嵌套？
13 // 3. 如何处理共享中断？
```

实现策略：

1. 先实现最基本的时钟中断处理
2. 逐步添加其他中断源支持
3. 考虑性能和可扩展性

任务4：实现上下文保存与恢复

参考xv6的kernelvec.S，理解：

1. 哪些寄存器必须保存？
 - 调用者保存寄存器 vs 被调用者保存寄存器
 - 临时寄存器的处理策略
 - CSR寄存器的保存需求
2. 栈的管理：
 - 中断栈的分配
 - 栈溢出检测
 - 多级中断的栈管理

实现挑战：

```
1 # 你的中断入口实现框架
2 kernelvec:
3     # 保存上下文
4     # 你需要决定：
```

```

5      # 1. 保存到哪里？内核栈？专用区域？
6      # 2. 保存哪些寄存器？
7      # 3. 如何快速保存和恢复？
8
9      # 调用C处理函数
10     call kerneltrap
11
12     # 恢复上下文并返回

```

任务5：实现时钟中断与调度

时钟中断处理：

1. 理解SBI时钟接口：

```

1      // 设置下次时钟中断时间
2      void sbi_set_timer(uint64 time);
3      // 获取当前时间
4      uint64 get_time(void);

```

2. 实现时钟中断处理函数：

```

1      void timer_interrupt(void) {
2          // 1. 更新系统时间
3          // 2. 处理定时器事件
4          // 3. 触发任务调度
5          // 4. 设置下次中断时间
6      }

```

调度器集成：

- 如何在时钟中断中触发调度？
- 调度的时机选择有什么考虑？
- 如何确保调度的原子性？

任务6：异常处理机制

异常类型理解：

1. 指令地址未对齐
2. 指令访问故障
3. 非法指令
4. 断点
5. 加载地址未对齐
6. 加载访问故障
7. 存储地址未对齐
8. 存储访问故障
9. 用户模式环境调用
10. 监督模式环境调用